

Executive Summary

Predicting the next viral slang

Project members: Alejandro Vargas-Altamirano, Hargun Bhatia, Jameson Auger, Karthik Srinivasan, Yasir Khan

Problem definition:

Internet slang changes incredibly fast, creating major blind spots for anyone trying to track consumer behavior and cultural trends. This project solves that problem with a data science pipeline that catches new slang early by measuring how often specific words appear compared to everyday language. By tracking comment sections across active online communities, the pipeline captures new words as they form and maps how they spread from group to group over time. While we started this project with the intention of predicting viral slang, we ended up doing much more than that. Because words form the basis of human communication, tracking how vocabulary shifts allows us to capture and predict major cultural phenomena. This gives brands, platforms, and teams tracking consumer behavior a powerful, real-time tool to anticipate societal shifts and connect with their audience long before a movement hits the mainstream.

Dataset curation:

To build a high-signal dataset without getting lost in the noise of the entire platform, we moved away from broad keyword searches and focused on monitoring specific, highly active creator communities. Using third-party analytics leaderboards, we manually curated a custom roster of channels that consistently generate the highest comment volumes and audience engagement within our target demographic. Identifying these high-activity hubs allowed us to focus our data collection exactly where new vocabulary originates. Once our target channels were selected, we built an automated ingestion pipeline using the official YouTube Data API to systematically pull upload playlists and extract thousands of top comments with exact timestamps.

After creating a basic tokenizer we were able to extract the individual words from the YouTube API comment data. After our initial tokenization step there were 104,102 unique tokens identified in the data. However, of these tokens, there were many words that appeared only a handful of times, extremely common words such as “the”, “a”, or “also”, and tokens that were simply artifacts of our tokenization process such as “didn”, “ve”, and “s”. After removing these low information tokens we were left with 11,156 words to include in our analysis.

Feature and target selection:

Our forecasting framework was built using the most recent 20 weeks of data split into a 17 week observation period followed by a 3 week prediction period. These splits were chosen to provide sufficient historical context as well as consistent data quality. We defined virality based on the relative frequency in the most recent 3 weeks compared to the relative frequency over the 17 week observation period. The idea of this definition is to capture when a word begins to spike in popularity as it becomes viral. In particular our target variable was defined as:

$$y = \log\left(\frac{\text{Relative frequency for final 3 weeks}}{\text{Relative frequency for previous 17 weeks}}\right)$$

Thus a large positive y-value indicates that a word is “going viral” in the last 3 weeks.

Two sets of outliers were identified in the target distribution, one at around $y=15$, corresponding to words which did not appear at all in the first 17 weeks, but did appear in the next 3 weeks, and one at around $y=-13$ corresponding to words which appeared in the first 17 weeks but did not appear in the next 3 weeks. Both of these outlier sets were removed. An argument could be made for including the second set of outliers as one would not know in advance if a word would be used at all in the next 3 weeks, but the models performed significantly worse with the outliers included.

To predict whether a word would go viral, we represented each word using its historical data from the 17 week observation period. Feature engineering was primarily guided by domain knowledge and sought to capture key aspects of a word’s trajectory including: popularity, engagement, use across different communities, and temporal trends. In particular, we collected both weekly features recorded for each of the 17 weeks in the observation period as well as features aggregated across the entire observation period. Weekly features included: number of comments containing word, number of likes received by comments containing word, number of channels containing comments with word, etc. Aggregate features included: total number of weeks word appeared, average number of uses per week, variation in use across weeks.

One limitation to our feature selection is that we intentionally included a broad set of features. As such there is a high likelihood of near or exact collinearity in some features, which could hurt the stability of linear models. A future step could be to perform more systematic feature analysis using techniques such as SHAP, regularization, or feature importance analysis to identify the important features and reduce redundancy.

Model selection and performance:

To compare the candidate models, we performed nested 5-fold cross-validation. Hyperparameters for Random Forest, XGBoost, Histogram-based Gradient Boosting, Extra Trees, Ridge, and Lasso were optimized using **RandomizedSearchCV** inside the inner folds,

while the outer folds provided unbiased estimates of generalization performance. Linear and Dummy Regressors served as untuned baselines.

Among all the models considered, **XGBoost** achieved the strongest overall performance, obtaining nested CV RMSE of **0.859 ± 0.020** , an R^2 score of **0.329**, Pearson correlation of **0.575**, and Spearman correlation of **0.415**. Compared with the dummy baseline, XGBoost reduced RMSE by approximately **18%**. The final model was retrained on the complete training set using the best hyperparameters identified during nested CV (489 trees, max_depth = 6, min_child_weight = 7, learning_rate = 0.0091, subsample = 0.7839, colsample = 0.79, reg_alpha = 0.0002, reg_lambda = 1.7249). Evaluation on the held-out test set gave an RMSE of **0.842** (a **17%** improvement over the dummy baseline), closely matching the nested CV estimate and indicating good generalization.

Since many practical applications require **identifying viral words**, we added a classification layer on top of the regression model. A word is defined as **viral** if its future average relative frequency at least doubles, i.e., $y > \log(2)$. We converted the regression predictions into binary decisions by selecting a prediction threshold using out-of-fold training predictions. We computed the Precision-Recall curve on the training folds, considered only positive thresholds achieving at least **80% precision** (since downstream applications benefit more from a small set of highly reliable viral predictions than a larger set containing many false positives), and selected the threshold with the highest recall (breaking ties using F1 score). This resulted in a prediction threshold of **0.657**.

We evaluated this classifier on the held-out-test set, yielding the following statistics:

Truly Viral Words	True Positives	False Positives	Precision	Recall	ROC-AUC	Average Precision
135	43	7	0.86	0.319	0.771	0.485

We also evaluated our model using **Top-K retrieval**. Among the model's highest-ranked predictions on the hold-out test set, **7 out of the top 10** and **28 out of the top 50** viral words were correctly identified. Many of these predictions correspond to recognizable gaming, anime, and sports terms such as **Rasengan, Ky, Kh4, Bolin, Ashe, Galio, Sanji, Kaka, and Zlatan**, illustrating the model's ability to identify emerging culturally relevant terms before they become widely popular.

Future steps:

Future work could collect data across social media platforms such as Reddit, TikTok, and X, as well as domains such as news, politics, gaming, and entertainment. Text processing could be

improved by using more robust tokenization to merge spelling variants and by tracking common slang phrases such as “no cap” and “let him cook” as single expressions.

A rolling weekly forecasting framework could use the features of each (word, week) pair to predict the word’s growth in the next week. Because most words remain stable or decline while only a small number grow rapidly, standard regression metrics may hide poor performance on viral words. Future models could therefore place greater weight on large future growth so that missing a viral word is penalized more heavily than making a small error on an ordinary word.

Finally, sudden viral growth is often driven by external events rather than past usage alone, such as game trailer releases, celebrity endorsements, and major news events. The model’s ability to detect newly viral terms could be improved by including external signals such as Google Trends and news activity.